

Data Science & Feature Engineering

Technical Report

*Part A: Conceptual Foundations & Section 15: Empirical Notebook Summary
(DPFE/final/a.ipynb)*

Part A: Conceptual Foundation

1. Short Notes

a) What is Data Analysis?

Data Analysis is the systematic process of inspecting, cleaning, transforming, and modeling raw data to discover actionable insights, draw logical conclusions, and support strategic decision-making.

- **Descriptive Analysis:** Answers 'What happened?' by summarizing historical data through aggregations, KPI dashboards, and summary statistics.
- **Diagnostic Analysis:** Answers 'Why did it happen?' using drill-down analytics, correlation matrices, and root-cause investigation.
- **Predictive Analysis:** Answers 'What is likely to happen?' leveraging statistical modeling and machine learning algorithms based on historical trends.
- **Prescriptive Analysis:** Answers 'What action should we take?' using optimization algorithms and decision simulation models.

b) How to Plan a Data Science Project

Structured project planning follows the CRISP-DM framework to ensure delivery alignment, data governance, and risk mitigation:

- **Business Understanding:** Align technical objectives with business KPIs (e.g., reducing credit default rates by 10%).
- **Data Audit & Feasibility:** Assess data availability, volume, privacy compliance (GDPR), and sample quality.
- **Infrastructure Setup:** Configure version control (Git), compute environments (Jupyter/GPUs), and data pipelines.
- **Iterative Modeling & Evaluation:** Establish simple baseline models before building complex architectures; evaluate using cross-validation.

c) How to Frame a Machine Learning Problem

Problem framing translates high-level business goals into well-defined machine learning tasks:

- **Task Translation:** Convert 'reduce customer churn' to a Binary Classification task predicting $P(\text{Churn}=1 \mid \text{Features})$.
- **Target & Feature Matrix:** Define label y and features X while ensuring strictly zero future data leakage.
- **Evaluation Metric Selection:** Choose metrics reflecting business cost (e.g., Precision/Recall/F1 for imbalanced fraud vs RMSE for pricing).
- **Operational Constraints:** Set latency budgets (real-time < 50ms vs batch scoring) and interpretability needs (SHAP/LIME).

2. Tensors Explanation & NumPy Implementations

A Tensor is a mathematical data structure generalizing scalars, vectors, and matrices to N dimensions (axes/rank).

```
import numpy as np

# Tensor Rank Hierarchy Examples
scalar = np.array(42)           # Rank 0 (0D)
vector = np.array([10.5, 20.0, 30.5]) # Rank 1 (1D)
matrix = np.array([[1, 2], [3, 4]])   # Rank 2 (2D)
tensor_3d = np.zeros((2, 3, 4))      # Rank 3 (3D - Time Series / Images)
tensor_4d = np.zeros((32, 224, 224, 3)) # Rank 4 (4D - CNN Image Batch)

# Essential Manipulations
matrix_reshaped = vector.reshape(3, 1) # Reshaping
matrix_mult = np.matmul(matrix, matrix) # Matrix Multiplication (@ operator)
```

15. Write a Report Summarizing Data Preprocessing & Feature Engineering (DPFE/final/a.ipynb)

This report presents an exact technical synthesis of the end-to-end data preprocessing and feature engineering workflow executed in **DPFE/final/a.ipynb**. Data was consolidated from multiple sources (CSV `main_transactions.csv`, JSON `customer_metadata.json`, SQLite DB `repayment_history.db`, and CSV `data.csv`) into a primary target DataFrame `df` with numeric columns `numeric_columns = ['age', 'annual_income', 'loan_amount', 'credit_score', 'repayment_history', 'transaction_count', 'spending_ratio']`.

- **Missing Value Strategies Used and Their Effectiveness**

In **a.ipynb**, missing data treatment was evaluated across 7 distinct approaches under Section 6:

- **1. SimpleImputer (Mean for Numerical):** Executed via `SimpleImputer(strategy='mean')` on `df_copy['age']`. Highly effective for baseline mean substitution, though it reduces overall feature variance.
- **2. SimpleImputer (Most Frequent for Categorical):** Executed via `SimpleImputer(strategy='most_frequent')` on `df['employment_type']` and `df['gender']`. Effectively imputes mode categories without introducing missing-class noise.
- **3. Missing Indicator + Random Sample Imputation:** Executed on `annual_income` by creating a binary column `df_sample['annual_income_missing_ind']` paired with random sampling from non-null entries. Highly effective at retaining variance while explicitly preserving missingness signal.
- **4. K-Nearest Neighbors Imputer (KNNImputer):** Executed via `KNNImputer(n_neighbors=5)` on `df_knn[numeric_columns]`. Distance-weighted neighbor averaging effectively preserved non-linear correlations across numerical variables.
- **5. MICE Algorithm (IterativeImputer - Production Choice):** Executed via `IterativeImputer(max_iter=10, random_state=42)` across `df[numeric_columns]`. This multivariate chained equations model proved most effective by modeling each feature against all other numerical features, delivering statistically superior imputed values.
- **6. Complete Case Analysis (CCA):** Evaluated by dropping rows/columns with excessive null values for reference analysis.

```
# Exact Code Implementation from a.ipynb (Section 6)
from sklearn.impute import KNNImputer, SimpleImputer
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# Categorical Most-Frequent Imputation
employment_imputer = SimpleImputer(strategy='most_frequent')
df['employment_type'] = employment_imputer.fit_transform(df[['employment_type']])

gender_imputer = SimpleImputer(strategy='most_frequent')
df['gender'] = gender_imputer.fit_transform(df[['gender']])

# Numerical MICE (IterativeImputer) Imputation
numeric_columns = ['age', 'annual_income', 'loan_amount', 'credit_score',
                   'repayment_history', 'transaction_count', 'spending_ratio']
iterative_imputer = IterativeImputer(max_iter=10, random_state=42)
df[numeric_columns] = iterative_imputer.fit_transform(df[numeric_columns])
```

• Outlier Handling Results

Outlier detection and treatment were performed in Section 7 of **a.ipynb** using four dedicated statistical techniques:

- **1. Z-Score Method:** Applied to `df['credit_score']` using `z_score = np.abs((df['credit_score'] - mean) / std)` to identify extreme deviations exceeding 3 standard deviations.

- **2. Interquartile Range (IQR) Capping:** Applied to `df['annual_income']` using `q1 = quantile(0.25)`, `q3 = quantile(0.75)`, `iqr = q3 - q1`. Extreme values were capped via `df['annual_income'].clip(lower=q1 - 1.5*iqr, upper=q3 + 1.5*iqr)`, removing extreme income tails without row loss.
- **3. Percentile Method:** Applied to `df['loan_amount']` using 1st percentile (`quantile(0.01)`) and 99th percentile (`quantile(0.99)`). Bounded via `df['loan_amount'].clip(lower=lower_percentile, upper=upper_percentile)`.
- **4. Winsorization Method:** Applied `scipy.stats.mstats.winsorize` to `df['spending_ratio']` with `limits=[0.01, 0.01]`, successfully stabilizing tail ratio variance.
- **5. Visualization Validation:** Rendered a 4x2 subplot grid of Seaborn boxplots comparing `df_copy[col]` vs treated `df[col]` across `main_cols = ['annual_income', 'loan_amount', 'credit_score', 'spending_ratio']`, confirming clean removal of outlier spikes.

```
# Exact Code Implementation from a.ipynb (Section 7)
from scipy.stats.mstats import winsorize

# IQR Capping on annual_income
q1 = df['annual_income'].quantile(0.25)
q3 = df['annual_income'].quantile(0.75)
iqr = q3 - q1
df['annual_income'] = df['annual_income'].clip(lower=q1 - 1.5*iqr, upper=q3 + 1.5*iqr)

# Percentile Clipping on loan_amount
lower_percentile = df['loan_amount'].quantile(0.01)
upper_percentile = df['loan_amount'].quantile(0.99)
df['loan_amount'] = df['loan_amount'].clip(lower=lower_percentile, upper=upper_percentile)

# Winsorization on spending_ratio
df['spending_ratio'] = winsorize(df['spending_ratio'], limits=[0.01, 0.01])
```

• Encoding Methods Applied to Categorical/Numerical Variables

Sections 8, 9, and 10 of **a.ipynb** cover variable type parsing, categorical encoding, and numerical feature discretization:

- **1. Date & Time Variable Encoding (Section 8.2):** Parsed `df['join_date'] = pd.to_datetime(...)` and extracted `df['join_year']`, `df['join_month']`, `df['join_day']`, and `df['join_weekday']`.
- **2. Ordinal Encoding (Section 9.1):** Executed `OrdinalEncoder(categories=[df['education_level'].unique()])` on `education_level` to preserve natural educational order.
- **3. Label Encoding (Section 9.2):** Executed `LabelEncoder()` on `df['gender']` to produce binary `df['gender_encoded']`.

- **4. One-Hot Encoding (Section 9.3):** Executed `pd.get_dummies(df['region'], prefix='region', drop_first=True)` and `OneHotEncoder(sparse_output=False, drop='first')` on `df[['loan_purpose']]` to eliminate dummy variable traps.
- **5. Equal-Width Binning (Section 10.1):** Executed `pd.cut()` on `df['annual_income']` and `df['repayment_history']` into 3 equal-width bins with labels=['Low', 'Medium', 'High'].
- **6. Binarization (Section 10.2):** Executed `Binarizer(threshold=700)` on `df[['credit_score']]` to generate `df['credit_score_binarized']`.
- **7. Quantile Binning (Section 10.3):** Executed `pd.qcut(df['transaction_count'], q=4, labels=['Low', 'Medium', 'High', 'Very High'])`.
- **8. K-Means Binning (Section 10.4):** Executed `KBinsDiscretizer(n_bins=4, encode='ordinal', strategy='kmeans')` on `df[['transaction_count']]` to create `df['transaction_count_kmeans_binned']`.

```
# Exact Code Implementation from a.ipynb (Sections 8, 9, 10)
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder, OneHotEncoder,
Binarizer, KBinsDiscretizer

# Date Feature Extraction
df['join_date'] = pd.to_datetime(df['join_date'], errors='coerce')
df['join_year'] = df['join_date'].dt.year
df['join_month'] = df['join_date'].dt.month
df['join_day'] = df['join_date'].dt.day

# Ordinal & Label Encoding
education_encoder = OrdinalEncoder(categories=[df['education_level'].unique()])
df['education_encoded'] = education_encoder.fit_transform(df[['education_level']])

gender_encoder = LabelEncoder()
df['gender_encoded'] = gender_encoder.fit_transform(df['gender'])

# One-Hot Encoding & Discretization
ohe = OneHotEncoder(sparse_output=False, drop='first')
loan_purpose_encoded = ohe.fit_transform(df[['loan_purpose']])

credit_score_bin = Binarizer(threshold=700)
df['credit_score_binarized'] = credit_score_bin.fit_transform(df[['credit_score']])

transaction_count_bin = KBinsDiscretizer(n_bins=4, encode='ordinal',
strategy='kmeans')
df['transaction_count_kmeans_binned'] =
transaction_count_bin.fit_transform(df[['transaction_count']])
```

• Scaling/Transformations Applied and Why

Sections 11 and 12 of **a.ipynb** implement comprehensive feature scaling and variance-stabilizing mathematical transformations:

- **1. StandardScaler (Section 11.1):** Executed via `StandardScaler().fit_transform(df[['annual_income', 'loan_amount']])` to yield mean=0 and std=1, required for distance-sensitive and gradient-descent models.
- **2. Normalizer (L2 Norm) (Section 11.2):** Executed via `Normalizer(norm='l2').fit_transform(df[numeric_columns])` for row-wise unit vector normalization.
- **3. MinMaxScaler (Section 11.3):** Executed via `MinMaxScaler().fit_transform(df[numeric_columns])` to bound features into [0, 1] range.
- **4. MaxAbsScaler (Section 11.4):** Executed via `MaxAbsScaler().fit_transform(df[numeric_columns])` to scale features by maximum absolute value into [-1, 1].
- **5. RobustScaler (Section 11.5):** Executed via `RobustScaler().fit_transform(df[numeric_columns])` using median and IQR to resist remaining outlier influence.
- **6. FunctionTransformer (Section 12.1):** Executed `FunctionTransformer(np.log1p)` on `spending_ratio` to produce `df['spending_ratio_log_transformed']` and `FunctionTransformer(np.sqrt)` for `spending_ratio_sqrt_transformed`, reducing right-skewness.
- **7. PowerTransformer (Box-Cox & Yeo-Johnson) (Section 12.2):** Executed `PowerTransformer(method='box-cox')` on `df[['loan_amount']]` to create `df['loan-amount_boxcox_transformed']` and `PowerTransformer(method='yeo-johnson')` on `df[['annual_income']]` to create `df['annual_income_yeojohnson_transformed']`, transforming non-normal features into Gaussian distributions.
- **8. Column Transformer (Section 12.3):** Executed `ColumnTransformer(transformers=[('num', StandardScaler(), ['annual_income', 'loan_amount']), ('cat', OneHotEncoder(drop='first'), ['employment_type', 'gender', 'region', 'loan_purpose'])])` to combine numeric scaling and categorical encoding into a single pipeline.

```
# Exact Code Implementation from a.ipynb (Sections 11, 12)
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler,
PowerTransformer, FunctionTransformer
from sklearn.compose import ColumnTransformer

# Function & Power Transformations
log_transformer = FunctionTransformer(np.log1p, validate=True)
df['spending_ratio_log_transformed'] =
log_transformer.fit_transform(df[['spending_ratio']])

pt_box = PowerTransformer(method='box-cox')
df['loan-amount_boxcox_transformed'] = pt_box.fit_transform(df[['loan_amount']])

pt_yeo = PowerTransformer(method='yeo-johnson')
df['annual_income_yeojohnson_transformed'] =
pt_yeo.fit_transform(df[['annual_income']])

# ColumnTransformer Composite Pipeline
ct = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), ['annual_income', 'loan_amount']),
```

```

        ('cat', OneHotEncoder(drop='first'), ['employment_type', 'gender', 'region',
        'loan_purpose']))
    ]
)
transformed_array = ct.fit_transform(df)

```

• Newly Engineered Features and Their Usefulness

Section 13 of **a.ipynb** constructs three domain-specific financial features to capture customer credit risk and behavior:

- **1. debt_to_income_ratio (Section 13.1):** Constructed via `df['debt_to_income_ratio'] = df['loan_amount'] / df['annual_income']`. Usefulness: Quantifies a customer's total financial leverage and repayment capacity. Higher ratios directly indicate elevated loan default risk.
- **2. avg_monthly_transactions (Section 13.2):** Constructed via `df['avg_monthly_transactions'] = df['transaction_count'] / 12`. Usefulness: Standardizes raw annual transaction frequency into a monthly activity cadence to evaluate customer account engagement level.
- **3. spending_to_income_ratio (Section 13.3):** Constructed via `df['spending_to_income_ratio'] = df['spending_ratio'] / df['annual_income']`. Usefulness: Measures relative discretionary spending against income, supporting credit scoring and risk limit assignment.

```

# Exact Code Implementation from a.ipynb (Section 13)
# 13.1 Debt-to-Income Ratio
df['debt_to_income_ratio'] = df['loan_amount'] / df['annual_income']

# 13.2 Average Monthly Transactions
df['avg_monthly_transactions'] = df['transaction_count'] / 12

# 13.3 Spending-to-Income Ratio
df['spending_to_income_ratio'] = df['spending_ratio'] / df['annual_income']

```

• Final Dataset Shape and Readiness for ML Modeling

The complete feature engineering pipeline in **a.ipynb** successfully converts raw multi-source data into a clean, model-ready feature matrix:

- **1. Data Completeness:** Achieved zero missing values across all numerical features (via MICE IterativeImputer) and categorical features (via SimpleImputer `most_frequent`).
- **2. Outlier Stabilization:** Variance in `annual_income`, `loan_amount`, and `spending_ratio` is bounded via IQR, Percentile clipping, and Winsorization without row loss.
- **3. Encoding & Dimensional Integrity:** Categorical columns (`education_level`, `gender`, `region`, `loan_purpose`) are fully encoded via Ordinal, Label, and One-Hot encoding with `drop='first'` preventing multicollinearity.

- **4. Gaussian Distribution Normality:** Skewed features are transformed via Box-Cox and Yeo-Johnson power transformations.
- **5. Production ML Readiness:** The processed dataset features high numerical stability, zero target leakage, normalized feature scales, and engineered financial risk indicators (debt_to_income_ratio, spending_to_income_ratio), making it fully prepared for machine learning model training (Logistic Regression, Random Forest, XGBoost, Neural Networks).